

Low-Level Design (LLD)

Professional-Skepticism Engine — module, schema & agent design

Field	Value	Field	Value
Project	Professional-Skepticism Engine	Document	Low-Level Design (LLD)
Version	1.1	Status	Reviewed — aligned with PRD v1.1
Owner	Capstone Team 01 (2 members)	Date	2026-06-25
Pattern	Multi-agent (debate + verify)	Stack	LangGraph · Claude · Streamlit · SQLite

1. Introduction

This LLD specifies the implementation-level design: repository layout, data models, per-agent contracts (model, temperature, prompt role, I/O schema, logic), the LangGraph wiring & routing, the RAG and connector designs, the database schema, configuration, error handling, and the test design.

2. Module Decomposition

```
skeptic-engine/  
  app/  
    state.py          # InvestigationState (TypedDict) + Pydantic models  
    graph.py          # LangGraph nodes, edges, routing, checkpointer  
    config.py         # thresholds, materiality, model map, round caps  
    llm.py            # Claude client + structured-output helper + retry  
    db.py             # SQLite: cases, reviews, audit_log (+ hash chain)  
  agents/  
    supervisor.py evidence.py challenger.py defender.py  
    adjudicator.py verifier.py review.py report.py  
  rag/  
    ingest.py store.py retrieve.py      # chunk -> embed -> hybrid search  
  connectors/  
    base.py fx.py registry.py cache.py  
  ui/streamlit_app.py  
  policies/ data/ tests/  
  Dockerfile requirements.txt README.md
```

3. Data Model

3.1 Pydantic models

```

class Evidence(BaseModel):
    policy: list[PolicyClause]          # id, version, text
    history: dict                       # vendor_prior_12m, account_mean, z_score
    external: dict                      # fx, vendor_master, ...
    citations: list[str]
    api_metadata: list[dict]           # source, status, latency_ms, cached

class ChallengerView(BaseModel):
    position: Literal["concern"]
    severity: Literal["low", "medium", "high"]
    claims: list[str]
    red_flags: list[str]

class DefenderView(BaseModel):
    position: Literal["legitimate_possible", "insufficient_basis"]
    arguments: list[str]
    conditions: list[str]
    confidence_in_legitimacy: float = Field(ge=0, le=1)

class Adjudication(BaseModel):
    risk_level: Literal["High", "Medium", "Low"]
    confidence: float = Field(ge=0, le=1)
    disposition: str
    rationale: str
    citations: list[str]

class Verification(BaseModel):
    passed: bool
    checked_claims: int; grounded: int; ungrounded: int
    issues: list[str]
    action: Literal["proceed", "revise"]

class ReviewEvent(BaseModel):
    reviewer_id: str
    action: Literal["approve", "reject", "reassign", "escalate"]
    comment: str | None; signature: str; timestamp: str

```

3.2 Shared state

```

class InvestigationState(TypedDict):
    case: dict
    plan: dict
    evidence: Evidence | None
    challenger_view: ChallengerView | None
    defender_view: DefenderView | None
    debate_transcript: Annotated[list[dict], add]
    adjudication: Adjudication | None
    verification: Verification | None
    routing: dict
    review_history: Annotated[list[ReviewEvent], add]
    audit_history: Annotated[list[dict], add]
    round: int                # debate/verify round counter (bounded)
    report: str | None

```

4. Agent Detailed Design

Agent (node)	Model · temp	I/O (Pydantic)	Core logic
supervisor	sonnet · 0.0	state → plan{}	read flags+amount; set rounds, materiality, specialists, route hint
evidence	sonnet · 0.0 (+tools)	case → Evidence	RAG retrieve + history query + FX/registry call; attach citations + api_metadata

Agent (node)	Model · temp	I/O (Pydantic)	Core logic
challenger	sonnet · 0.5	case+evidence → ChallengerView	argue concern ONLY; cite red flags from evidence ids
defender	sonnet · 0.5	case+evidence → DefenderView	argue legitimacy ONLY; isolated from challenger output
adjudicator	sonnet · 0.0	evidence+views → Adjudication	weigh both; assign risk+confidence+disposition; must cite
verifier	sonnet · 0.0	adjudication+evidence → Verification	map each claim to a citation; set passed/action
review	human	bundle → ReviewEvent	interrupt(); resume with signed decision
report	haiku/code	all → CaseFile	render MD/HTML/JSON case file

4.1 Node signature & prompt contract (pattern)

```
def challenger(state: InvestigationState) -> dict:
    ev = state["evidence"]
    view = call_structured(
        model=MODEL_MAP["challenger"], temperature=0.5,
        system=PROMPTS["challenger"],          # "You are the skeptic. Argue ONLY the
                                                # case for concern. Cite evidence ids."
        user=render(state["case"], ev),
        schema=ChallengerView, retries=3)
    return {"challenger_view": view,
            "debate_transcript": [{"role": "challenger", "content": view.model_dump()}]}
```

5. Orchestration (graph.py)

```
g = StateGraph(InvestigationState)
for name, fn in NODES: g.add_node(name, fn)
g.add_edge(START, "supervisor"); g.add_edge("supervisor", "evidence")
g.add_edge("evidence", "challenger"); g.add_edge("evidence", "defender") # parallel
g.add_edge(["challenger", "defender"], "adjudicator") # fan-in
g.add_edge("adjudicator", "verifier")
g.add_conditional_edges("verifier", route_after_verify, # revise loop (bounded)
                        {"revise": "adjudicator", "gate": "confidence_gate"})
g.add_conditional_edges("confidence_gate", route_after_gate,
                        {"auto": "report", "human": "review"})
g.add_conditional_edges("review", route_after_review,
                        {"approve": "report", "reject": "report", "revise": "adjudicator"})
g.add_edge("report", "audit"); g.add_edge("audit", END)
app = g.compile(checkpointer=SqliteSaver.from_conn_string("checkpoints.db"))
```

5.1 Routing functions

```
def route_after_verify(s):
    if s["verification"].action == "revise" and s["round"] < MAX_DEBATE_ROUNDS:
        s["round"] += 1; return "revise"
    return "gate"

def route_after_gate(s):
    a = s["adjudication"]; amt = s["case"]["amount_usd"]
    if a.confidence >= AUTO and s["verification"].passed and a.risk_level=="Low" \
        and amt < MATERIALITY: return "auto"
    return "human"
```

5.2 Human-in-the-loop

```
def review(state):
    d = interrupt({"transcript": state["debate_transcript"],
                  "adjudication": state["adjudication"].model_dump()})
    return {"review_history": [ReviewEvent(**d)],
            "audit_history": [audit_event("review", d)]}
# UI resumes: app.invoke(Command(resume=decision), config={"configurable":{"thread_id":tid}})
```

6. RAG Design

- **Ingest:** policies (PDF/MD) → split by clause (≈200–400 tokens, 15% overlap) → embed → upsert to Chroma with metadata {doc, clause_id, version, effective_date}.
- **Retrieve:** build query from case (account, vendor, amount, flags) → hybrid (BM25 + dense, RRF) → top-k=8 → cross-encoder re-rank → top-n=4.
- **Contract:** retrieve(query, k=8, n=4) -> list[PolicyClause] with ids the Adjudicator must cite.

7. Connector Design

```
class Provider(Protocol):
    def fetch(self, case) -> dict: ... # returns an evidence fragment

class FXProvider: # connectors/fx.py
    def fetch(self, case):
        key = f"{case['currency']}:{case['date']}"
        if key in cache: return cache[key]
        try:
            r = httpx.get(f"https://api.frankfurter.app/{case['date']}",
                          params={"from":case["currency"],"to":"USD"}, timeout=3)
            ev = build(r.json()); cache[key]=ev; return ev
        except Exception:
            return {"status":"unavailable"} # evidence missing -> lowers confidence
```

8. Database Schema (SQLite)

Table	Columns
cases	case_id PK · txn_id · payload_json · status · created_at
reviews	id PK · case_id FK · reviewer_id · action · comment · signature · ts
audit_log	id PK · case_id · event · payload_json · hash_prev · hash · ts
checkpoints	(managed by LangGraph SqliteSaver — thread_id · state blob)

audit_log is append-only; each row stores hash = sha256(hash_prev + payload) for tamper-evidence.

9. Configuration (config.py)

```
AUTO, REVIEW = 0.90, 0.70
MATERIALITY = 25_000
MAX_DEBATE_ROUNDS = 2
MODEL_MAP = {"supervisor":"sonnet","evidence":"sonnet","challenger":"sonnet",
              "defender":"sonnet","adjudicator":"sonnet","verifier":"sonnet",
              "report":"haiku"}
TEMPS = {"challenger":0.5,"defender":0.5,"adjudicator":0.0,"verifier":0.0,"supervisor":0.0}
```

10. Error Handling & Edge Cases

Case	Handling
Claude 429 / 5xx / timeout	Exponential backoff retry (3x) in call_structured; then mark node failed → DLQ
External API down	Timeout + cache fallback; record evidence as unavailable; lower confidence
Verifier fails repeatedly	Bounded re-runs (MAX_DEBATE_ROUNDS); then force route = manual
Malformed LLM output	Pydantic validation error → 1 reprompt with the error → else manual
Reviewer never responds	Case stays paused in checkpoint (durable); SLA escalation (future)

11. Test Design

- **Unit (per agent):** mock the LLM (canned structured output), assert schema + routing behaviour.
- **RAG:** retrieval precision on a seeded policy set; citation-id presence.
- **Verifier:** inject ungrounded claims → assert `action=="revise"`.
- **Integration:** full case run; assert auto-clear vs human path; restart mid-debate resumes.
- **Eval harness:** A/B single-prompt vs crew on a labelled set → FP/FN deltas (PRD §17).